14 haftalık yol haritasında 10. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

# HAFTA 10

# Diziler I: Bitişik Bellek ve Tek Boyutlu Diziler

Bildirim ve ilk değer; indis ve adres hesabı; dizi desenleri; frekans dizisi ve histogram; dizileri fonksiyonlara aktarma; sınır dışı erişim ve güvenlik

MODÜL V · VERİ TOPLULUKLARI VE BELLEK · H10–H12

HAFTANIN BÜYÜK SORUSU

*Yüz öğrencinin notunu yüz ayrı değişkende mi tutacağız ve dizinin sınırını bir adım aşmak neden bir güvenlik felaketine dönüşebilir?*

**Dr. Öğr. Üyesi Mehmet Sevri**
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

# İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları ..... 2
2. Geçen Haftadan: Çağrı Yığınından Bitişik Belleğe ..... 3
3. Kavramsal Anlatım ..... 4
3.1 Neden dizi? ..... 4
3.2 Bildirim, boyut ve ilk değer ..... 4
3.3 İndisler 0'dan başlar: adres hesabı ..... 5
3.4 Döngüyle dolaşma, kapasite ve sizeof ..... 6
3.5 Değişken uzunluklu diziler (VLA) hakkında not ..... 7
3.6 Dizileri fonksiyonlara aktarmak ..... 7
3.7 Kopyalama, karşılaştırma ve birleştirme ..... 8
3.8 Paralel diziler ..... 9
3.9 Sınır dışı erişim: tanımsız davranış ve güvenlik ..... 9
3.10 Kaputun altında: sıralı erişim ve önbellek ..... 10
4. Çözümlü Örnekler: Örnek Merdiveni ..... 10
4.1 ★ basamaklar: dizi ile ilk adımlar (Örnek 1–4) ..... 11
4.2 Örnek 5 ★★ — en büyük, en küçük ve indisleri ..... 12
4.3 Örnek 6 ★★ — değer var mı, nerede? (doğrusal tarama) ..... 13
4.4 Amiral örnek — Örnek 7 ★★: diziyi yerinde ters çevirme ..... 13
4.5 Örnek 9 ★★ — 6000 zar atışı: frekans dizisi ve histogram ..... 15
4.6 Amiral örnek — Örnek 12 ★★★: araya ekleme ve silme ..... 17
4.7 Örnek 13 ★★★ — sınır dışı yazma deneyi ..... 19
4.8 Örnek 14 ★★★ — Eratosthenes Kalburu (meydan okuma) ..... 21
5. Hata Avcısı ..... 21
6. Mühendis Gözüyle: Sınırı Programın Kendisi Bilmeli ..... 23
7. Mini-OBS Adımı: v0.10 — Paralel Dizilerde Kayıt, İstatistik ve Histogram ..... 24
8. Alıştırmalar ve Atölye ..... 25
9. Kendini Değerlendirme ..... 28
10. AP-2 Köprüsü ve Merak Kutusu ..... 28
11. Özet, Kavram Haritası ve Terimler ..... 29
12. Kaynaklar ..... 30

# 1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

**Yüz öğrencinin notunu yüz ayrı değişkende mi tutacağız ve dizinin sınırını bir adım aşmak neden bir güvenlik felaketine dönüşebilir?**

H9'a kadar her değeri ayrı bir değişkende tuttuk. Mini-OBS v0.9 öğrencileri **saklamadan**, akış hâlinde işliyordu: her not girildiğinde toplamı ve sayacı güncelliyor, sonra o notu unutuyordu. Bu yüzden "ortalamanın üstündeki öğrenciler kimler?" sorusunu yanıtlayamıyordu: ortalama ancak bütün notlar okunduktan sonra bilinir, o anda ise eski notlar yoktur. Bu haftanın yanıtı **dizi**dir (array): aynı tipten çok sayıda değeri **tek bir adla**, bellekte **art arda** (bitişik) saklayan ve her birine **indisle** (index) ulaşan yapı.

> GERÇEK DÜNYADAN · **Heartbleed (CVE-2014-0160): uzunluk alanına güvenmenin bedeli**
>
> TLS'in **Heartbeat** (kalp atışı) uzantısı, bağlantının hâlâ canlı olduğunu sınamak içindir: bir taraf bir yük (payload) ve bu yükün uzunluğunu gönderir, karşı taraf aynı yükü geri yollar. Uzantı Şubat 2012'de RFC 6520 ile tanımlandı ve OpenSSL'e 14 Mart 2012'de yayımlanan 1.0.1 sürümüyle girdi (Durumeric vd., 2014). OpenSSL'in gerçekleştirimi, istekte **bildirilen** uzunluğu gerçekte gelen yükün uzunluğuyla **karşılaştırmıyordu**: küçük bir yük gönderip büyük bir uzunluk bildiren saldırgan, sunucunun belleğinde yükün **ardından gelen** baytları yanıtta geri alıyordu. Uzunluk alanı 2 bayt olduğundan bir istekte en çok $2^{16}$ bayt (~64 KB) okunabiliyordu ve istek tekrarlanabiliyordu (OpenSSL Project, 2014; Durumeric vd., 2014).
>
> Açığı Google Security'den Neel Mehta ve Codenomicon şirketinden bir ekip birbirinden bağımsız olarak buldu (Durumeric vd., 2014); OpenSSL, 7 Nisan 2014'te düzeltilmiş 1.0.1g sürümüyle birlikte duyurdu. Etkilenen sürümler 1.0.1–1.0.1f ve 1.0.2-beta1 idi (OpenSSL Project, 2014). NVD açığı bir **tampon sınırı ötesini okuma** (buffer over-read) olarak tanımlar ve özel anahtarların okunabildiğini not eder (National Institute of Standards and Technology [NIST], 2014). Durumeric vd. (2014), duyuru anında Alexa'nın en popüler bir milyon sitesindeki HTTPS sunucularının tahminen %24–55'inin açık olduğunu ölçtü. Düzeltme tek bir **sınır denetimiydi**: bildirilen uzunluk gerçek yükü aşıyorsa istek yanıtlanmadan atılır.
>
> Bu haftanın dersi: bir dizinin sınırını **programın kendisi** bilmeli ve denetlemelidir; C bunu sizin yerinize yapmaz. Bölüm 3.9'da aynı hatayı, kendi dizimizin içinde kalan güvenli bir benzetimle (`H10_K11_heartbeat_benzetimi.c`) üreteceğiz.

### Öğrenme çıktıları

**ÖÇ 10.1** Tek boyutlu dizileri bildirir, ilk değer atar ve indisler üzerinden erişir. *(Bölüm 3.2–3.4, 4.1; Örnek 1–2)*

**ÖÇ 10.2** Dizilerin bellekte bitişik yerleşimini ve indis–adres ilişkisini (taban adres + i × eleman boyutu) açıklar. *(Bölüm 3.3, 4.7; Örnek 13)*

**ÖÇ 10.3** Temel dizi işleme desenlerini (toplam, ortalama, en büyük ve konumu, sayma, frekans, ters çevirme, kaydırma, kopyalama) uygular. *(Bölüm 3.7, 4.1–4.6, 4.8; Örnek 3–10, 12, 14)*

**ÖÇ 10.4** Dizileri boyut bilgisiyle birlikte fonksiyonlara aktarır. *(Bölüm 3.6, 4.5; Örnek 11)*

**ÖÇ 10.5** Dizi sınırı aşımının (out-of-bounds) sonuçlarını ve güvenlik riskini açıklar. *(Bölüm 3.9, 4.7; Örnek 13)*

## 2. Geçen Haftadan: Çağrı Yığınından Bitişik Belleğe

H9'da kapsamı, `static` ömrü, çağrı yığınını ve özyinelemeyi gördük. Üç geri çağırma sorusu (programlar `kodlar/quiz/` klasöründe):

> TAHMİN ET · **Geri çağırma: üç soru**
>
> - **S1 (static yerel).** `H10_Q01_static_sayac.c`: `say()` iki kez boşa, üçüncüsü `printf` içinde çağrılıyor. Ne yazdırılır?
> - **S2 (çağrı yığını).** `H10_Q02_cagri_yigini.c`: `main` → `a` → `b`. Çıktı sırası nedir; yığında aynı anda en çok kaç çerçeve olur?
> - **S3 (özyineleme).** `H10_Q03_basamak_toplami.c`: `topla(2026)` ne döndürür, temel durum hangisi?

**Yanıtlar (gerçek çıktılar).** S1: `3`: `static int n` yalnızca bir kez sıfırlanır, her çağrıda bir artar. S2: "m1 a1 b a2 m2"; `b` çalışırken yığında `main`, `a` ve `b` olmak üzere **üç** çerçeve vardır. S3: `10` (6 + 2 + 0 + 2); temel durum `n == 0`.

**Önceki haftalarla bağ.** Bu hafta üç açık soruyu kapatıyoruz: `H07_10_onluk_ikili.c`'nin **ters sırada** üretilen bitleri bir dizide saklanıp sondan başa yazılacak (Bölüm 4.1); `H08_11_zar_frekans.c`'nin altı ayrı sayacı ve `switch`'i frekans dizisiyle tek satıra inecek (Örnek 9); `H08_10_takas_denemesi.c`'de çalışmayan takasın aksine dizi elemanlarının fonksiyon içinde değiştirilebildiğini görecek ve nedenine ilk kez bakacağız (Bölüm 3.6; tam yanıt H12). H9'un yığını da geri dönüyor: yerel diziler yığındadır ve sınır dışı yazmanın etkisi bu yerleşime bağlıdır (Örnek 13). H7'nin √n asallık testi (`H07_11_asal_karekok_sure.c`) Örnek 14'te Eratosthenes Kalburu ile karşılaştırılacak.

# 3. Kavramsal Anlatım

## 3.1 Neden dizi?

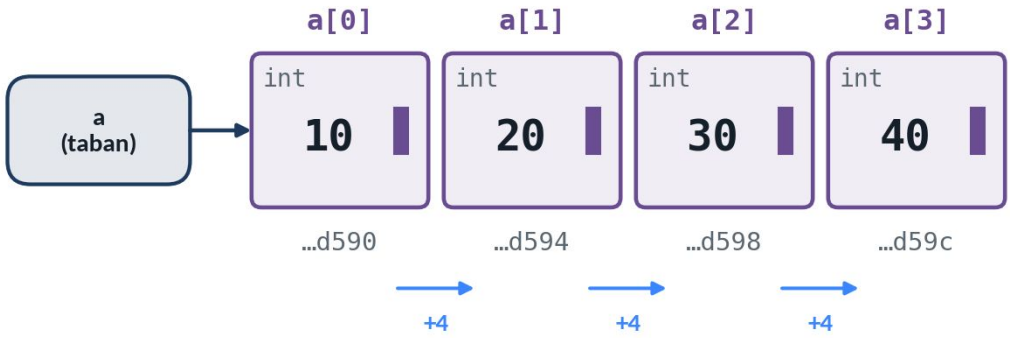
Beş öğrencinin notlarını ayrı değişkenlerde tutalım ve ortalamanın üstündekileri yazdıralım. `H10_K01_ayri_degiskenler.c` aynı işi iki kez yapar. Birinci yöntemde beş ayrı değişken (`not1` … `not5`), beş terimli bir toplama ifadesi ve **beş ayrı** `if` vardır. İkinci yöntem tek bir dizi ve iki döngüdür:

İki yöntem aynı çıktıyı verir ("Diziyle : ortalama 73.80, üstündekiler: 85 90"). Ama öğrenci sayısı 100 olsa ilk yöntemde 100 değişken, 100 `if` ve uzun bir toplama ifadesi gerekirdi; ikincisinde yalnızca `N` değişir. Dizinin gücü, **indisin bir değişken olabilmesidir**: `notlar[i]` ifadesindeki `i` döngüyle 0'dan N − 1'e kadar dolaşır.

**Analoji: numaralı dolaplar.** Bir spor salonundaki dolap sırasını düşünün: sıranın **tek bir adı** vardır, dolaplar **aynı boyutta** ve **yan yana**dır, her birine **numarasıyla** ulaşılır. `int a[4]` dört eşit (4 baytlık) dolaptan oluşan bir sıradır; `a[2]` "a sırasının 2 numaralı dolabı" demektir. Dolapların bitişik olması, herhangi birinin yerini **hesaplayabilmemizi** sağlar (Bölüm 3.3; Şekil 10.1).

**Şekil 10.1**
*Dizi = numaralı dolap şeridi: üstte indis, içte değer, altta adres (gerçek adresler H10_K03_adresler.c'den)*

```
int a[4] = {10, 20, 30, 40};  →  4 × 4 bayt = 16 bayt, bitişik
```




```
taban = &a[0] = 0x7ffd2232d590 (gerçek çıktı, H10_K03_adresler.c)
```

## 3.2 Bildirim, boyut ve ilk değer

Dizi bildirimi `tip ad[BOYUT];` biçimindedir: `int notlar[5];` beş `int`lik yer ayırır. Boyut sıfırdan büyük bir **tamsayı sabit ifadesidir**; sabiti `#define N 5` ile tek yerde tanımlamak iyi alışkanlıktır (ileride

değişecek tek satır). Bildirimde **ilk değer listesi** (initializer list) verilebilir. `H10_K02_bildirim_ilk_deger.c` beş biçimi dener:

**H10_K02_bildirim_ilk_deger.c · satır 17–21**

```
17      int a[5] = {1, 2, 3, 4, 5};         /* tam liste */
18      int b[5] = {1, 2};                  /* kısmi: kalanlar 0 */
19      int c[5] = {0};                     /* hepsi 0 */
20      int d[] = {4, 8, 15, 16, 23, 42};   /* boyut listeden: 6 */
21      int e[6] = {[2] = 7, [5] = 9};      /* C99: belirlenmiş ilk değer */
```

Gerçek çıktı: “a (tam liste): 1 2 3 4 5”, “b (kısmi): 1 2 0 0 0”, “c ({0}): 0 0 0 0 0”, “d (boyut: 6): 4 8 15 16 23 42”, “e ([2], [5]): 0 0 7 0 0 9”, “g (global): 0 0 0 0 0”.

Kurallar: (1) Liste kısaysa **kalan elemanlar 0** olur (`b`); bu yüzden `= {0}` bütün diziyi sıfırlamanın kısa yoludur (`c`). (2) Boyut yazılmazsa derleyici listedeki eleman sayısını kullanır (`d`: 6). (3) C99’dan beri **belirlenmiş ilk değer** (designated initializer) ile yalnızca bazı indisler verilebilir (`e`). (4) Dosya kapsamındaki (global) ya da `static` diziler kendiliğinden sıfırla başlar (`g`); fonksiyon içindeki ilk değersiz bir dizinin elemanları ise **belirsizdir** (indeterminate): o bellekte daha önce ne kaldıysa o görünebilir (Hata Avcısı 2) (ISO/IEC, 2018). Boyuttan fazla eleman vermek (`int f[3] = {1, 2, 3, 4};`) bir **kısıt ihlalidir**: derleyici bir tanı vermek zorundadır, ama GCC 13.3 ve Clang 18 yalnızca uyarır (“warning: excess elements in array initializer”) ve fazla değeri atar; `-pedantic-errors` ile aynı tanı derleme hatasıdır (`H10_K15_fazla_ilk_deger.c`). Uyarıyı hata gibi ele alın (King, 2008, Bölüm 8).

## 3.3 İndisler 0’dan başlar: adres hesabı

Dizinin elemanları bellekte **bitişik** durur: ilk elemanın adresine **taban adres** (base address) denir ve i. elemanın adresi

**adres(a[i]) = taban + i × sizeof(a[0])**

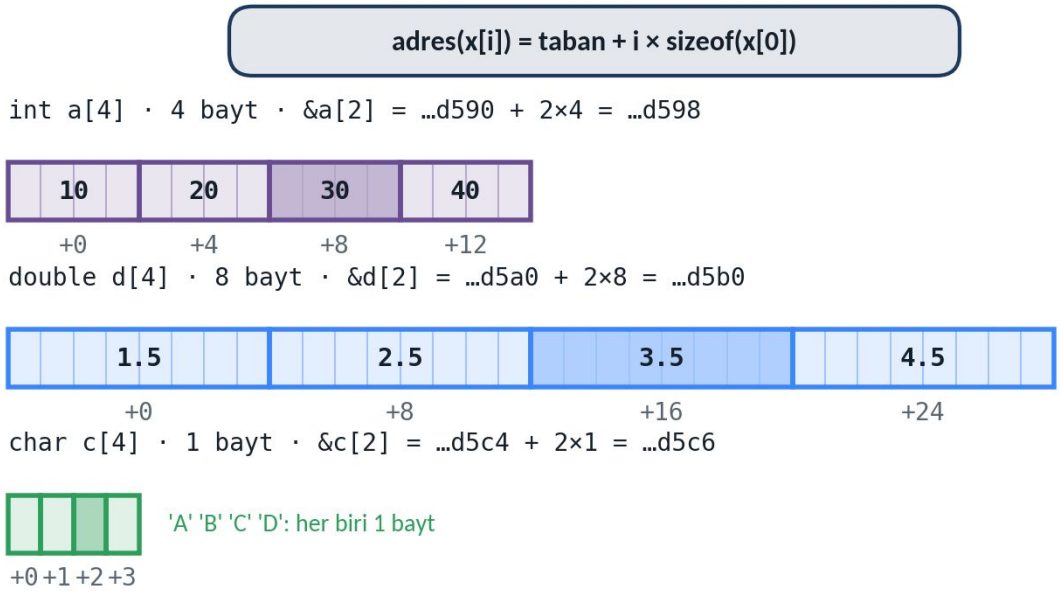
formülüyle hesaplanır. İndis aslında **tabandan uzaklıktır** (ofset): ilk eleman tabandan 0 eleman uzaktadır, bu yüzden indisi 0’dır. Son eleman ise N − 1’dir; `a[N]` dizinin **dışındadır**. `H10_K03_adresler.c` aynı dört elemanı `int a[4]`, `double d[4]` ve `char c[4]` dizilerinde tutar ve bir döngüde `&a[i]`, `&d[i]`, `&c[i]` adreslerini `%p` ile yazdırır:

**GERÇEK ÇIKTI (1. ÇALIŞTIRMA)**

```
$ ./adresler      # 1. çalıştırma
sizeof: int 4, double 8, char 1 bayt
i=0  &a[i]=0x7ffd2232d590  &d[i]=0x7ffd2232d5a0  &c[i]=0x7ffd2232d5c4
i=1  &a[i]=0x7ffd2232d594  &d[i]=0x7ffd2232d5a8  &c[i]=0x7ffd2232d5c5
i=2  &a[i]=0x7ffd2232d598  &d[i]=0x7ffd2232d5b0  &c[i]=0x7ffd2232d5c6
i=3  &a[i]=0x7ffd2232d59c  &d[i]=0x7ffd2232d5b8  &c[i]=0x7ffd2232d5c7
a = 0x7ffd2232d590 (dizinin adı = &a[0])
a[2] = 30, *(a + 2) = 30
```

Adresler eleman boyutu kadar artar: `int` dizisinde 4’er (0x7ffd2232d590 → 0x7ffd2232d594), `double` dizisinde 8’er (0x7ffd2232d5a0 → 0x7ffd2232d5a8), `char` dizisinde 1’er bayt. Örnek: `&a[2]` = 0x7ffd2232d590 + 2 × 4 = 0x7ffd2232d598 (çıktıyla aynı). İkinci çalıştırmada taban adres değişti (0x7fffb2113a60): işletim sistemi yığının yerini her çalıştırmada rastgele seçer (ASLR, H3); ama **elemanlar arasındaki fark değişmez**, çünkü formül yalnızca tabana ve eleman boyutuna bağlıdır (Şekil 10.2).

**Şekil 10.2**
*adres = taban + i × sizeof(eleman): int (4), double (8) ve char (1 bayt) dizilerinde her kutu bir bayt (gerçek adresler)*




> **KAPUTUN ALTINDA · a[i] ≡ *(a + i): ilk bakış (ayrıntısı H12'de)**
>
> İfadelerde dizinin **adı**, ilk elemanının adresine dönüşür: çıktıdaki `a = …` satırı `&a[0]` ile aynı adresi gösterir. C standardı `a[i]` ifadesini **tanım olarak** `*(a + i)` biçiminde verir: "a adresinden i eleman ileri git, oradaki değeri al" (ISO/IEC, 2018, 6.5.2.1; Kernighan & Ritchie, 1988, Bölüm 5.3). Çıktıdaki `a[2] = 30, *(a + 2) = 30` satırı bunu gösterir; `+ i`'nin bayt değil **eleman** saydığına dikkat edin (derleyici i'yi eleman boyutuyla çarpar).
>
> Derleyici de böyle görür: Bölüm 3.6'daki `H10_K12_const_ihlali.c` derlenirken GCC, `a[i] = 0;` satırını hata iletisinde `*(a + (sizetype)((long unsigned int)i * 4))` olarak yazar: indis × 4 bayt. Bu yüzden indisle erişim, eleman sayısından bağımsız olarak hızlıdır (bir toplama + bir bellek erişimi). `&`, `*` ve işaretçi aritmetiği H12'nin konusudur.

## 3.4 Döngüyle dolaşma, kapasite ve sizeof

Diziyle çalışmanın standart kalıbı `for (int i = 0; i < N; i++)` döngüsüdür: tam N tur döner ve i 0'dan N − 1'e gider (Kernighan & Ritchie, 1988, Bölüm 1.6). Örnek 1 beş not okur ve yazar:

**H10_01_dizi_oku_yaz.c · satır 14–22**

```
14      int notlar[N];                      /* 5 int: notlar[0] .. notlar[4] */
15
16      printf("%d not girin: ", N);
17      for (int i = 0; i < N; i++) {
18          if (scanf("%d", &notlar[i]) != 1) {   /* &: i. elemanın adresi */
19              printf("Geçersiz girdi.\n");
20              return 1;
21          }
22      }
```

Girdi “72 85 64 90 58” için program her satıra bir eleman yazar: “notlar[0] = 72” … “notlar[4] = 58” (gerçek çıktı).

`scanf`’e bir değişkenin **adresi** verilir (H3); dizi elemanında bu `&notlar[i]`’dir (Hata Avcısı 5). **Kapasite ve dolu eleman sayısı ayrıdır:** çoğu programda kaç değer geleceği önceden bilinmez; diziye yeterli bir kapasite verilir (`#define KAPASITE 100`) ve kaç elemanın dolu olduğu ayrı bir değişkende (`n`) tutulur. Döngüler `i < n` ile döner, kullanıcıdan gelen `n` ise **kapasiteyle denetlenir** (Örnek 3: `n < 1 || n > KAPASITE` ise reddet).

**Eleman sayısını derleyiciye hesaplatmak.** `sizeof(dizi)` dizinin toplam bayt sayısını, `sizeof(dizi[0])` bir elemanın bayt sayısını verir; bölümleri eleman sayısıdır. Hesap derleme zamanında yapılır ve ilk değer listesi değişince kendiliğinden güncellenir. H9’daki makro tekniğiyle (parantezlere dikkat!) tek satıra indirilebilir:

**H10_K04_sizeof.c · satır 13**

```
13  #define ELEMAN_SAYISI(d) (sizeof(d) / sizeof((d)[0]))   /* parantezler! */
```

Gerçek çıktı (x86-64 Linux, LP64): “double b[10]: sizeof(b) = 80, sizeof(b[0]) = 8, eleman = 10”; `int` için 40 / 4 = 10, `char` için 10 / 1 = 10.

**Uyarı:** bu yöntem yalnızca dizinin **bildirildiği** yerde çalışır. Fonksiyon parametresi olarak yazılan bir “dizi” aslında işaretçidir; orada `sizeof` 8 verir (Bölüm 3.6, Hata Avcısı 3).

## 3.5 Değişken uzunluklu diziler (VLA) hakkında not

C99, boyutu **çalışma zamanında** belirlenen yerel dizilere izin verdi: `int kareler[n];` (variable-length array, VLA). `H10_K05_vla.c` boyutu kullanıcıdan alır; `sizeof` da çalışma zamanında hesaplanır:

**H10_K05_vla.c · satır 26–31**

```
26      int kareler[n];                    /* VLA: boyut n, çalışma zamanında */
27      for (int i = 0; i < n; i++) {
28          kareler[i] = i * i;
29      }
30      printf("kareler[%d] = %d\n", n - 1, kareler[n - 1]);
31      printf("sizeof(kareler) = %zu bayt (çalışma zamanında)\n", sizeof(kareler));
```

Girdi 6 için gerçek çıktı: “kareler[5] = 25”, “sizeof(kareler) = 24 bayt (çalışma zamanında)”.

VLA, C99’da **zorunlu**, C11’den beri **isteğe bağlı** (conditional) bir özelliktir: desteklemeyen bir gerçekleştirim `__STDC_NO_VLA__` makrosunu 1 olarak tanımlar (ISO/IEC, 2018, 6.10.8.3). GCC VLA’yı destekler ve bu makroyu tanımlamaz (`gcc -dM -E` ile denetlendi: 0 eşleşme). Microsoft’un C derleyicisi (MSVC) ise `/std:c11` ve `/std:c17` kipinde `__STDC_NO_VLA__`’yı 1 tanımlar, yani VLA desteklemez (Microsoft, t.y.). `-Wvla` seçeneği VLA kullanımını uyarıya çevirir: “warning: ISO C90 forbids variable length array ‘kareler’ [-Wvla]”. n çok büyükse dizi yığını (Linux’ta tipik 8 MB, H9) taşırır; SEI CERT C kuralı ARR32-C bu yüzden VLA boyutunun denetlenmesini ister (Carnegie Mellon University Software Engineering Institute [SEI], 2016). Bu derste **sabit kapasiteli** diziler kullanıyoruz; çalışma zamanında boyutlanan güvenli diziler H12’nin (`malloc`) konusudur.

## 3.6 Dizileri fonksiyonlara aktarmak

Diziyi alan bir fonksiyonun parametresi `int a[]` biçiminde yazılır ve **eleman sayısı ayrı bir parametredir**: `int dizi_toplam(const int a[], int n)`. Çağrıda yalnızca dizinin adı yazılır:

`dizi_toplam(sayilar, n)`. Diziyi yalnızca **okuyan** fonksiyonlar parametreyi `const` ile işaretler; bu bir sözdür ve derleyici bu sözü denetler. `H10_K12_const_ihlali.c` bilerek `const` dizinin bir elemanına yazar (`a[i] = 0;`):

GERÇEK DERLEYİCİ İLETİSİ (DOSYA DERLENMEZ)

```
16:14: error: assignment of read-only location '*(a + (sizetype)((long unsigned int)i
* 4))'
```

**Neden boyut ayrı?** Fonksiyona dizinin kopyası değil, **ilk elemanının adresi** gider. Parametredeki `int a[]` yazımı aslında `int *a`'dır (bir işaretçi); dizinin kaç elemanlı olduğu bilgisi bu adreste yoktur. Bu yüzden fonksiyon içinde `sizeof(a)` dizinin değil işaretçinin boyutunu (8 bayt) verir; GCC bunu `-Wsizeof-array-argument` ile uyarır (Hata Avcısı 3).

KAPUTUN ALTINDA · **Dizi elemanları neden fonksiyon içinde değişebilir? (H8'deki takas sorusuna ara ipucu)**

H8'de `takas(a, b)` `main`'deki değişkenleri değiştirememişti: fonksiyona değerlerin **kopyaları** gidiyordu (`H08_10_takas_denemesi.c`). `H10_K06_dizi_adresi.c` aynı karşılaştırmayı dizi ve sayıyla yapar: `eleman_takas(a, 0, 1)` dizinin iki elemanını değiştirir ve `main` bunu görür; `sifirla(s)` ise yalnızca kendi kopyasını sıfırlar.

`H10_K06_dizi_adresi.c · satır 30–36`

```
30  void eleman_takas(int dizi[], int i, int j)
31  {
32      printf("takas: dizi  = %p   (main'deki &a[0] ile aynı)\n", (void *)dizi);
33      int gecici = dizi[i];
34      dizi[i] = dizi[j];
35      dizi[j] = gecici;
36  }
```

Gerçek çıktı: "main : &a[0] = 0x7ffe390bec1c &s = 0x7ffe390bec18", "takas: dizi = 0x7ffe390bec1c (main'deki &a[0] ile aynı)", "sifirla: &x = 0x7ffe390bebfc (main'deki &s'den farklı)", "sonra: a = {9, 7, 0}, s = 7".

Fonksiyondaki `dizi` adresi `main`'deki `&a[0]` ile **aynıdır**: iki fonksiyon aynı bellek kutularına bakar. `sifirla`'daki `&x` ise `&s`'den farklıdır: ayrı bir kutu, ayrı bir çerçeve. Demek ki bir fonksiyona bir değişkenin **adresini** verebilseydik o değişkeni de değiştirebilirdi; bunun aracı **işaretçidir** ve takas sorusunun tam çözümü H12'dedir.

## 3.7 Kopyalama, karşılaştırma ve birleştirme

C'de diziler bir bütün olarak **atanamaz** ve `==` ile **karşılaştırılamaz**: `b = a;` derleme hatasıdır, `a == b` ise içerikleri değil adresleri karşılaştırır (Hata Avcısı 4). Üç iş de eleman eleman bir döngüyle yapılır:

**H10_K09_kopyala_karsilastir.c · satır 23–34**

```
23      for (int i = 0; i < N; i++) {       /* kopyalama: eleman eleman */
24          b[i] = a[i];
25      }
26
27      bool esit = true;                   /* karşılaştırma: ilk farkta dur */
28      for (int i = 0; i < N; i++) {
29          if (a[i] != b[i]) {
30              esit = false;
31              break;
32          }
33      }
34      printf("a ile b %s\n", esit ? "eşit" : "farklı");
```

Gerçek çıktı: “a ile b eşit”, “birleşik : 3 1 4 1 5 9 2 6”.

Karşılaştırma döngüsü ilk farkta `break` ile çıkar (Örnek 6’daki erken çıkış). Birleştirmede ikinci dizinin elemanları `birlesik[N + i]` konumuna, yani birinci dizinin **hemen ardına** yazılır. Standart kütüphanede bayt düzeyinde karşılıkları vardır: `memcpy(d, c, sizeof(c))` ve `memcmp(c, d, sizeof(c))` (`<string.h>`; uzunluk **bayt** cinsindendir, eleman değil); aynı dosyanın son satırı “memcmp(c, d) = 0 (0: aynı)” yazar.

## 3.8 Paralel diziler

Bir öğrencinin numarası, vizesi ve finali farklı tiptedir; tek bir dizide tutulamaz. H10’un çözümü **paralel dizilerdir** (parallel arrays): `no[i]`, `vize[i]` ve `final[i]` **aynı indiste aynı öğrenciye** aittir. En yüksek finalin indisini bulup o indisle diğer dizileri okuruz (`H10_K10_paralel_diziler.c`). Sınırlılığı da aynı programda görülür: bir kaydı silmek, taşımak ya da sıralamak bütün dizilere **aynı işlemi** uygulamayı gerektirir; biri unutulursa kayıtlar kayar:

GERÇEK ÇIKTI

```
$ ./paralel_diziler
En yüksek final: 95 (no 1047, vize 90)
Silmeden sonra (vize[] ve final[] unutuldu):
  no 1045  vize 70  final 85
  no 1047  vize 45  final 48
  no 1048  vize 90  final 95
```

Silmeden sonra 1047 numaralı öğrenci 1046’nın notlarını (45, 48), 1048 ise 1047’ninkileri (90, 95) almış görünüyor. Mini-OBS v0.10 dört paralel dizi kullanır (Bölüm 7). Bilgileri bir arada tutan **yapı** (`struct Ogrenci`) H14’te bu sorunu kökten çözecek.

## 3.9 Sınır dışı erişim: tanımsız davranış ve güvenlik

C, dizi erişiminde **sınır denetimi yapmaz**: `a[i]` yalnızca taban + i × boyut adresine gider. i dizinin dışını gösteriyorsa (i < 0 ya da i ≥ N) okuma ve yazma **tanımsız davranıştır** (undefined behavior): standart hiçbir sonuç garanti etmez (ISO/IEC, 2018, 6.5.6 ve J.2). Program çökebilir, başka bir değişkeni sessizce bozabilir ya da “çalışıyor” görünebilir; en tehlikelisi sonuncusudur: hata testte görünmez, sahada ortaya çıkar (yazma deneyi: Örnek 13). Burada sınır dışı **okumanın** sonucunu, Heartbleed’in özünü taşıyan güvenli bir benzetimle görelim. `H10_K11_heartbeat_benzetimi.c`’de “sunucu belleği” **tek bir** `char` dizisidir: önce 3 baytlık yük (`KUS`), ardından başka veriler. Hatalı fonksiyon istemcinin bildirdiği

uzunluğa güvenir; aynı dosyadaki `yanit_duzeltilmis` ise kopyalamadan önce `if (bildirilen > gercek)` diye sorar ve isteği yanıtlamadan atar:

**H10_K11_heartbeat_benzetimi.c · satır 46–53**

```
46  /* HATALI: istemcinin bildirdiği uzunluğa GÜVENİR (denetim yok). */
47  int yanit_hatali(const char bellek[], int bildirilen, char yanit[])
48  {
49      for (int i = 0; i < bildirilen; i++) {
50          yanit[i] = bellek[i];
51      }
52      return bildirilen;
53  }
```

GERÇEK ÇIKTI

```
$ ./heartbeat
İstek: yük "KUS" (3 bayt), bildirilen uzunluk 3
  hatalı     : "KUS" (3 bayt)
  düzeltilmiş: "KUS" (3 bayt)
İstek: yük "KUS" (3 bayt), bildirilen uzunluk 31
  hatalı     : "KUS|oturum=7f3a91|parola=Rize53" (31 bayt)
  düzeltilmiş: istek atıldı (0 bayt)
```

Bildirilen uzunluk 31 olunca hatalı yanıt, yükün ardındaki oturum anahtarını ve parolayı da içerir. Benzetimde okuma kendi dizimizin içinde kaldığı için davranış tanımlıdır (`main`, bildirilen uzunluğu `sizeof(bellek) - 1` ile sınırlar); gerçek açıkta ise kopyalama **tamponun dışına** taşıyor ve sunucu sürecinin belleğinde yükün ardında ne varsa (oturum çerezleri, parolalar, özel anahtarlar) yanıta giriyordu (Durumeric vd., 2014). Düzeltilmiş fonksiyon, OpenSSL 1.0.1g'deki gibi, bildirilen uzunluk gerçek yükü aşıyorsa isteği yanıtlamaz.

### 3.10 Kaputun altında: sıralı erişim ve önbellek

> KAPUTUN ALTINDA · **Diziyi baştan sona dolaşmak neden hızlıdır?**
>
> İşlemci belleği bayt bayt değil, **önbellek satırı** (cache line) denen bloklar hâlinde getirir; bu makinede satır 64 bayttır (`getconf LEVEL1_DCACHE_LINESIZE`), yani 16 `int`. a[0] okunurken a[0]…a[15] birlikte önbelleğe gelir: sonraki 15 erişim RAM'e gitmeden **isabet** (hit) olur. Dizinin bitişik olması ve döngünün indisleri sırayla artırması bu yüzden **önbellek dostudur** (cache-friendly). Atlayarak erişmek (a[0], a[16], a[32], …) her seferinde yeni bir satır ister. Bu ilkeye **uzamsal yerellik** (spatial locality) denir (Bryant & O'Hallaron, 2016).
>
> Bu hafta yalnızca sezgiyi kuruyoruz; etkisini H11'de büyük bir matrisi satır satır ve sütun sütun dolaşarak `clock()` ile **ölçeceğiz**.

## 4. Çözümlü Örnekler: Örnek Merdiveni

Örnekler kolaydan zora sıralıdır (`kodlar/H10_01_…` – `H10_14_…`). Hepsi `gcc -std=c17 -Wall -Wextra -pedantic -g` ile **sıfır uyarıyla** derlenir; Örnek 13 dışındakiler AddressSanitizer ve UBSan ile de temiz çalışır; bütün çıktılar gerçektir. Örnek 7 ve 12 standart çözümlü örnek şablonunun tamamıyla işlenmiştir.

## 4.1 ★ basamaklar: dizi ile ilk adımlar (Örnek 1–4)

**Örnek 1 ★** (beş not oku–yazdır) Bölüm 3.4'te incelendi; geçersiz girdide ("72 85 x") program "Geçersiz girdi." yazıp durur. **Dizili çözüm: H7'nin tersten yazdırma sorunu.** `H07_10_onluk_ikili.c` ardışık 2'ye bölmenin kalanlarını ters sırada üretiyordu. Kalanları bir dizide **saklarsak** sorun kalmaz: `H10_K07_ikili_dizi.c` bitleri `bitler[0]`'dan başlayarak yazar, sonra diziyi **sondan başa** yazdırır.

**H10_K07_ikili_dizi.c · satır 26–35**

```
26      do {                                /* n = 0 için de bir bit: 0 */
27          bitler[adet] = sayi % 2;        /* en sağdaki bit ilk gelir */
28          adet++;
29          sayi /= 2;
30      } while (sayi > 0);
31
32      printf("%d (onluk) = ", n);
33      for (int i = adet - 1; i >= 0; i--) {   /* SONDAN BAŞA yazdır */
34          printf("%d", bitler[i]);
35      }
```

Gerçek çıktılar: "13 (onluk) = 1101 (ikili), 4 bit", "2026 (onluk) = 11111101010 (ikili), 11 bit", "0 (onluk) = 0 (ikili), 1 bit".

### Örnek 2 ★ — toplam ve ortalama

**H10_02_toplam_ortalama.c · satır 13–20**

```
13      int notlar[] = {72, 85, 64, 90, 58, 77, 81};   /* boyut listeden: 7 */
14      int n = (int)(sizeof(notlar) / sizeof(notlar[0]));
15
16      int toplam = 0;                     /* toplayıcı: 0'dan başlar */
17      for (int i = 0; i < n; i++) {
18          toplam += notlar[i];
19      }
20      double ortalama = (double)toplam / n;          /* tamsayı bölmesi değil */
```

Gerçek çıktı: "Dizinin boyutu: 28 bayt, eleman: 7", "Toplam  : 527", "Ortalama : 75.29".

H6'nın toplayıcı deseni aynen geçerli; değişen tek şey, değerlerin girdiden değil **diziden** gelmesidir. `(double)toplam / n` ile tamsayı bölmesinden kaçınılır (H4). **Değiştir:** en düşük notu atıp kalanların ortalamasını alın.

### Örnek 3 ★ — ortalamanın üstündekiler: neden dizi gerekli?

**H10_03_ortalama_ustu.c · satır 36–43**

```
36      int adet = 0;
37      for (int i = 0; i < n; i++) {       /* 2. geçiş: saklananları karşılaştır */
38          if (notlar[i] > ortalama) {
39              printf("  %d. öğrenci: %d\n", i + 1, notlar[i]);
40              adet++;
41          }
42      }
43      printf("Ortalamanın üstünde: %d not\n", adet);
```

Birinci geçiş (`for` içinde `scanf` ile oku, `notlar[i]`'ye sakla, `toplam`'a ekle) Örnek 1 ve 2'nin birleşimidir; yukarıda ikinci geçiş görülüyor. Girdi "6  72 85 64 90 58 77" için gerçek çıktı: "6 not: 72 85 64 90 58 77", "Ortalama: 74.33", "2. öğrenci: 85", "4. öğrenci: 90", "6. öğrenci: 77", "Ortalamanın üstünde: 3 not".

Bu problem **iki geçiş** ister: ortalama bütün notlar okunmadan bilinemez, karşılaştırma ise ortalamayı gerektirir. Birinci geçişte değerleri **saklamadan** ikinci geçiş yapılamaz. Mini-OBS v0.9'un bu soruyu yanıtlayamamasının nedeni buydu. Bütün notlar eşit olursa (ikinci kayıtlı çalıştırma: 50 50 50) hiçbiri ortalamadan **kesin büyük** değildir, sonuç 0 not.

### Örnek 4 ★ — bir değer kaç kez geçiyor? (sayma)

**H10_04_deger_sayma.c · satır 22–28**

```
22      int sayac = 0;
23      for (int i = 0; i < n; i++) {
24          if (atislar[i] == aranan) {     /* koşul: eşit mi? */
25              sayac++;
26          }
27      }
28      printf("%d değeri %d atışta %d kez geçti.\n", aranan, n, sayac);
```

Gerçek çıktılar: "6 değeri 12 atışta 5 kez geçti.", "9 değeri 12 atışta 0 kez geçti.". Sayma deseni dizinin **tamamını** dolaşır (erken çıkış yok); hiç geçmeyen değer için sonuç 0'dır, hata değil.

## 4.2 Örnek 5 ★★ — en büyük, en küçük ve indisleri

**H10_05_en_buyuk_indis.c · satır 16–27**

```
16      int enb = 0;                        /* en büyüğün İNDİSİ */
17      int enk = 0;                        /* en küçüğün İNDİSİ */
18      for (int i = 1; i < n; i++) {       /* 0. eleman zaten aday */
19          if (puanlar[i] > puanlar[enb]) {
20              enb = i;
21          }
22          if (puanlar[i] < puanlar[enk]) {
23              enk = i;
24          }
25      }
26      printf("En büyük: %d (indis %d, %d. sırada)\n", puanlar[enb], enb, enb + 1);
27      printf("En küçük: %d (indis %d, %d. sırada)\n", puanlar[enk], enk, enk + 1);
```

**Tablo 10.1**
*Örnek 5'in izleme tablosu (puanlar = {64, 91, 77, 91, 58, 85}); değişen hücreler vurgulu*

| i | puanlar[i] | puanlar[i] > puanlar[enb] ? | enb | enk |
|---|---|---|---|---|
| başta | – | – | 0 | 0 |
| 1 | 91 | 91 > 64 ? evet | **1** | 0 |
| 2 | 77 | 77 > 91 ? hayır | 1 | 0 |
| 3 | 91 | 91 > 91 ? hayır | 1 | 0 |
| 4 | 58 | 58 > 91 ? hayır | 1 | **4** |
| 5 | 85 | 85 > 91 ? hayır | 1 | 4 |

Gerçek çıktı: “En büyük: 91 (indis 1, 2. sırada)”, “En küçük: 58 (indis 4, 5. sırada)”.

İki ayrıntı: (1) Değer değil **indis** saklanır; değer her zaman `puanlar[enb]` ile okunabilir, tersi mümkün değildir. (2) `>` kullanıldığı için eşitlikte (i = 3’teki ikinci 91) aday değişmez: sonuç **ilk** indistir. `>=` yazılsaydı son indis (3) olurdu. Başlangıç değeri 0. elemandır, 0 sayısı değil: bütün notlar negatif olsa da doğru çalışır. **Değiştir:** en büyüğün kaç kez geçtiğini de sayın.

## 4.3 Örnek 6 ★★ — değer var mı, nerede? (doğrusal tarama)

**H10_06_dogrusal_tarama.c · satır 24–31**

```
24      int konum = -1;                    /* "bulunamadı" anlamında */
25      int karsilastirma = 0;
26      for (int i = 0; i < n; i++) {
27          karsilastirma++;
28          if (numaralar[i] == aranan) {
29              konum = i;
30              break;                     /* erken çıkış: ilk eşleşme yeter */
31          }
```

Gerçek çıktılar: “1012 bulundu: indis 1 (2 karşılaştırma)”, “1090 bulundu: indis 5 (6 karşılaştırma)”, “1111 yok: konum = -1 (6 karşılaştırma)”.

**Doğrusal tarama** (linear search) baştan sona sırayla bakar ve **ilk** eşleşmede `break` ile çıkar; bulunamazsa konum, geçerli bir indis olamayacak **−1** kalır. 1012 dizide iki kez geçtiği hâlde ilk konum (1) döner ve yalnızca 2 karşılaştırma yapılır; bulunamayan değer için 6 elemanın 6’sına da bakılır. Karşılaştırma sayısının en iyi ve en kötü durumdaki biçimsel çözümlemesi AP-2’nin konusudur.

## 4.4 Amiral örnek — Örnek 7 ★★: diziyi yerinde ters çevirme

**1. Problem.** Bir dizinin elemanlarını, ikinci bir dizi kullanmadan **yerinde** (in place) ters sıraya çevirin: {10, 20, …, 70} → {70, 60, …, 10}. **2. Çözümleme.** Girdi n elemanlı bir `int` dizisi; çıktı aynı dizinin ters hâli. Ek bellek yalnızca bir `gecici` değişken. Sınır durumlar: n = 0 ve n = 1 (hiçbir şey yapılmamalı), n çift ya da tek (tekte ortadaki eleman yerinde kalır).

**Tablo 10.2**
*Örnek 7’nin IPO çözümlemesi*

| GİRDİ | İŞLEM | ÇIKTI |
|---|---|---|
| • n elemanlı dizi a (n ≥ 0) | • İki uç: i = 0, j = n − 1<br>• i < j iken a[i] ↔ a[j] takas; i++, j-- | • Aynı dizi, ters sırada<br>• Takas sayısı: n / 2 |

**3. Algoritma (iki uç tekniği).** `i ← 0; j ← n − 1; i < j olduğu sürece: geçici ← a[i]; a[i] ← a[j]; a[j] ← geçici; i ← i + 1; j ← j − 1.` Uçlar ortada buluşunca durulur. Koşul `i < j`’dir, `i < n` değil: dizinin sonuna kadar gitmek her çifti **iki kez** takas eder ve dizi başa döner (Kendini sına, S5). **4. C kodu.**

**H10_07_ters_cevir.c · satır 23–32**

```
23      int i = 0;                          /* sol uç */
24      int j = n - 1;                      /* sağ uç */
25      while (i < j) {                     /* uçlar ortada buluşana kadar */
26          printf("  takas: a[%d]=%d <-> a[%d]=%d\n", i, a[i], j, a[j]);
27          int gecici = a[i];              /* üç atamalı takas */
28          a[i] = a[j];
29          a[j] = gecici;
30          i++;
31          j--;
32      }
```

**5. Açıklama.** Satır 25: döngü koşulu iki indisin karşılaştırmasıdır. Satır 27–29: H3'teki üç atamalı takas, bu kez dizi elemanlarıyla. `gecici` döngü gövdesinde bildirildiği için her turda yeniden oluşur (blok kapsamı, H9). H12'de aynı takas işaretçilerle bir fonksiyona taşınacak. **6. İzleme tablosu** (Tablo 10.3; görsel sürümü sunumda ve web sayfasındaki animasyonda):

**Tablo 10.3**
*Örnek 7'nin izleme tablosu: her turda bir takas, i ve j birer adım yaklaşır*

| Tur | i (sonra) | j (sonra) | takas | dizi |
|---|---|---|---|---|
| başta | 0 | 6 | – | 10 20 30 40 50 60 70 |
| 1 | 1 | 5 | **a[0]=10 ↔ a[6]=70** | 70 20 30 40 50 60 10 |
| 2 | 2 | 4 | **a[1]=20 ↔ a[5]=60** | 70 60 30 40 50 20 10 |
| 3 | 3 | 3 | **a[2]=30 ↔ a[4]=50** | 70 60 50 40 30 20 10 |
| son | 3 | 3 | i < j yanlış: dur | 70 60 50 40 30 20 10 |

**7. Örnek çalıştırma.** Gerçek çıktının ilk ve son satırları: "Önce : 10 20 30 40 50 60 70", "Sonra : 70 60 50 40 30 20 10"; aradaki üç "takas:" satırı Tablo 10.3'nin takas sütunudur.

**8. Test tablosu.** Algoritmayı fonksiyona taşıyan `H10_K13_ters_cevir_test.c` sınır durumları dener (Tablo 10.4):

**Tablo 10.4**
*Örnek 7'nin test tablosu (H10_K13_ters_cevir_test.c'nin gerçek çıktısı)*

| n | Girdi | Gerçek çıktı | Takas | Beklenen |
|---|---|---|---|---|
| 0 | (boş) | (boş) | 0 | 0 takas · ✓ |
| 1 | 1 | 1 | 0 | 0 takas · ✓ |
| 2 | 1 2 | 2 1 | 1 | 1 takas · ✓ |
| 5 | 1 2 3 4 5 | 5 4 3 2 1 | 2 | 2 takas · ✓ |
| 6 | 1 2 3 4 5 6 | 6 5 4 3 2 1 | 3 | 3 takas · ✓ |

n = 0'da `j = -1` olur ve `i < j` baştan yanlıştır: döngü hiç dönmez, **dizinin dışına erişilmez**. **9. Değiştir–genişlet:** (a) Yalnızca a[s..e] aralığını ters çeviren `ters_aralik(a, s, e)` yazın. (b) Bir dizinin **palindrom** olup olmadığını aynı iki uç düşüncesiyle, takas yapmadan denetleyin (AP-2'de "iki işaretçi" yöntemi).

### Örnek 8 ★★ — bir konum sola döndürme

**H10_08_sola_dondur.c · satır 25–29**

```
25          int ilk = a[0];                 /* 1) ilk elemanı kenara al */
26          for (int i = 0; i < N - 1; i++) {
27              a[i] = a[i + 1];            /* 2) soldan sağa: bir sola kaydır */
28          }
29          a[N - 1] = ilk;                 /* 3) kenardakini sona koy */
```

Gerçek çıktı: “Başta: 1 2 3 4 5 6”, “1. dönüş: 2 3 4 5 6 1”, “2. dönüş: 3 4 5 6 1 2”, “3. dönüş: 4 5 6 1 2 3”.

Döndürme üç adımdır: ilk eleman kenara alınır, diğerleri **soldan sağa** birer sola kaydırılır, kenardaki sona konur. İç döngü `i < N - 1` ile durur: son turda `a[N - 2] = a[N - 1]` yapılır; `i < N` yazılsaydı `a[N]` okunurdu (bir-fark hatası). Kenara alınmazsa ilk eleman kaybolur. Üç dönüşten sonra dizi 4 5 6 1 2 3’tür; N dönüşte başa döner. **Değiştir:** sağa döndürme (Alıştırma 10.8).

## 4.5 Örnek 9 ★★ — 6000 zar atışı: frekans dizisi ve histogram

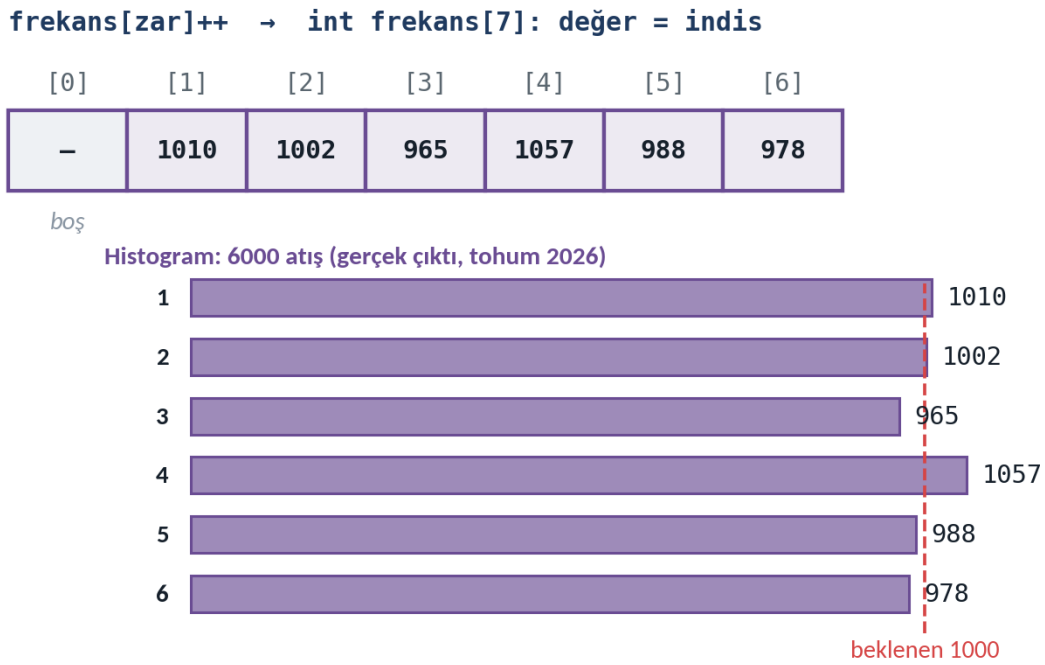
**H10_09_zar_histogram.c · satır 25–30**

```
25      int frekans[7] = {0};               /* [1]..[6] kullanılır, [0] boş */
26
27      for (int atis = 0; atis < ATIS; atis++) {
28          int zar = rand() % 6 + 1;       /* 1..6 */
29          frekans[zar]++;                 /* DEĞER = İNDİS */
30      }
```

Program sabit tohumla derlendi: `gcc -std=c17 -Wall -Wextra -pedantic -g -DSABIT_TOHUM=2026 H10_09_zar_histogram.c -o zar_histogram`. Her yıldız 20 atıştır; 1. yüzün satırı 50 yıldızdır (1010 / 20).

Ana fikir: zarın **değeri** sayacın **indisi**dir. `frekans[zar]++` satırı, H8’deki altı ayrı sayacı ve yedi dallı `switch`’i (`H08_11_zar_frekans.c`) tek satıra indirir. `frekans[7]` bilinçli bir seçimdir: `[1]..[6]` kullanılır, `[0]` boş kalır; böylece `zar - 1` dönüşümü gerekmez. Sayaçlar `= {0}` ile sıfırlanmalıdır (Hata Avcısı 2). Yüzlerin sayıları 1000 çevresinde dalgalanır (1010, 1002, 965, 1057, 988, 978; toplam 6000): rastgelelik (Şekil 10.3). Kayıtlı çıktının yeniden üretilebilmesi için program `-DSABIT_TOHUM=2026` ile derlendi (H6, H8); tohumsuz derlenirse her çalıştırmada farklıdır.

**Şekil 10.3**
*Frekans dizisinden histogram çubuklarına: frekans[1..6] ve 6000 atışın gerçek dağılımı*




## Örnek 10 ★★ — harf notu dağılımı (frekans + karar)

**H10_10_harf_dagilimi.c · satır 28–37**

```
28      for (int i = 0; i < n; i++) {
29          int y = (int)(0.40 * vize[i] + 0.60 * final[i] + 0.5);
30          int k = HARF - 1;               /* final barajı: FF */
31          if (final[i] >= 50) {
32              k = 0;
33              while (y < alt_sinir[k]) {  /* ilk uygun sınırı bul */
34                  k++;
35              }
36          }
37          sayac[k]++;                     /* harf indisinin sayacı */
```

Gerçek çıktı (15 öğrenci; her * bir öğrenci): AA 3, BA 1, BB 1, CB 2, CC 2, DC 3, DD 1, FD 0, FF 2.

Kurallar Mini-OBS ile aynıdır: %40 vize + %60 final, `(int)(ort + 0.5)` ile yuvarlama, final < 50 ise **FF** (baraj); diğer harfler RTEÜ tablosundan (AA 90–100, BA 81–89, BB 73–80, CB 66–72, CC 60–65, DC 50–59, DD 40–49, FD 30–39, FF 0–29). Bu sürümde `else-if` merdiveni yerine harf sınırları bir **dizide** durur: `while (y < alt_sinir[k]) k++;` ilk uygun sınırı bulur (y = 77 için 77 < 90, 77 < 81, 77 ≥ 73 → k = 2, BB). `alt_sinir[8] = 0` olduğundan döngü her zaman durur. Harf adları iki paralel `char` dizisinden yazılır: `ilk[k]` ve `ikinci[k]`.

## Örnek 11 ★★ — dizi fonksiyonları

**H10_11_dizi_fonksiyonlari.c · satır 15–18**

```
15  bool dizi_oku(int a[], int n);
16  void dizi_yazdir(const int a[], int n);
17  int dizi_toplam(const int a[], int n);
18  int dizi_max(const int a[], int n);
```

GERÇEK ÇIKTI

```
$ ./dizi_fonksiyonlari
Kaç sayı (1-50)? 6
6 sayı: 12 7 25 3 25 9
[12, 7, 25, 3, 25, 9]
Toplam = 81, en büyük = 25
```

Dört fonksiyonun hepsi diziyi ve **eleman sayısını** ayrı ayrı alır. `dizi_oku` diziyi doldurduğu için parametresi `const` değildir; diğer üçü yalnızca okur. `dizi_max`'in ön koşulu n ≥ 1'dir: başlangıç değeri `a[0]`'dır, "küçük bir sayı" değil (tek eleman −4 olduğunda da doğru). `main` artık yalnızca fonksiyon adlarından oluşan bir "içindekiler" sayfası gibi okunur (H8). Mini-OBS v0.10'un `dizi_ortalama`, `en_buyuk_indis`, `esik_ustu_say` ve `standart_sapma` fonksiyonları aynı kalıptadır.

## 4.6 Amiral örnek — Örnek 12 ★★★: araya ekleme ve silme

**1. Problem.** Kapasitesi 6 olan bir dizide 5 eleman var. Belirli bir konuma eleman **ekleyen** ve belirli bir konumdaki elemanı **silen**, sıralamayı koruyan iki fonksiyon yazın. **2. Çözümleme.** Ekleme için kapasitede yer olmalı ve 0 ≤ konum ≤ n olmalı (konum = n sona eklemektir); silme için 0 ≤ konum < n. Fonksiyon çağıranın `n`'ini değiştiremez (değerle aktarım, H8): **yeni n'yi döndürür**, geçersiz istekte −1 döndürür.

**Tablo 10.5**
*Örnek 12'nin IPO çözümlemesi*

| GİRDİ | İŞLEM | ÇIKTI |
|---|---|---|
| • dizi a, n, kapasite<br>• konum (ve eklenecek değer) | • Denetim: yer var mı, konum geçerli mi?<br>• Ekleme: konumdan sonrasını SAĞA kaydır<br>• Silme: konumdan sonrasını SOLA kaydır | • Güncellenmiş dizi<br>• Yeni n ya da −1 (hata) |

**3. Algoritma.** Ekleme: `i ← n; i > konum iken: a[i] ← a[i−1]; i ← i−1` (**sondan başa**), sonra `a[konum] ← değer`. Silme: `i ← konum; i < n−1 iken: a[i] ← a[i+1]; i ← i+1` (**baştan sona**). Kaydırma yönü kritiktir: sağa kaydırma baştan sona yapılırsa her eleman bir sonrakinin üstüne yazılır ve ilk değer bütün diziye yayılır (`H10_K08_kaydirma_yonu.c`; Şekil 10.4). **4. C kodu.**

**H10_12_ekle_sil.c · satır 39–49**

```
39  int ekle(int a[], int n, int kapasite, int konum, int deger)
40  {
41      if (n >= kapasite || konum < 0 || konum > n) {
42          return -1;
43      }
44      for (int i = n; i > konum; i--) {   /* SONDAN BAŞA: sağa kaydır */
45          a[i] = a[i - 1];
46      }
47      a[konum] = deger;
48      return n + 1;
49  }
```

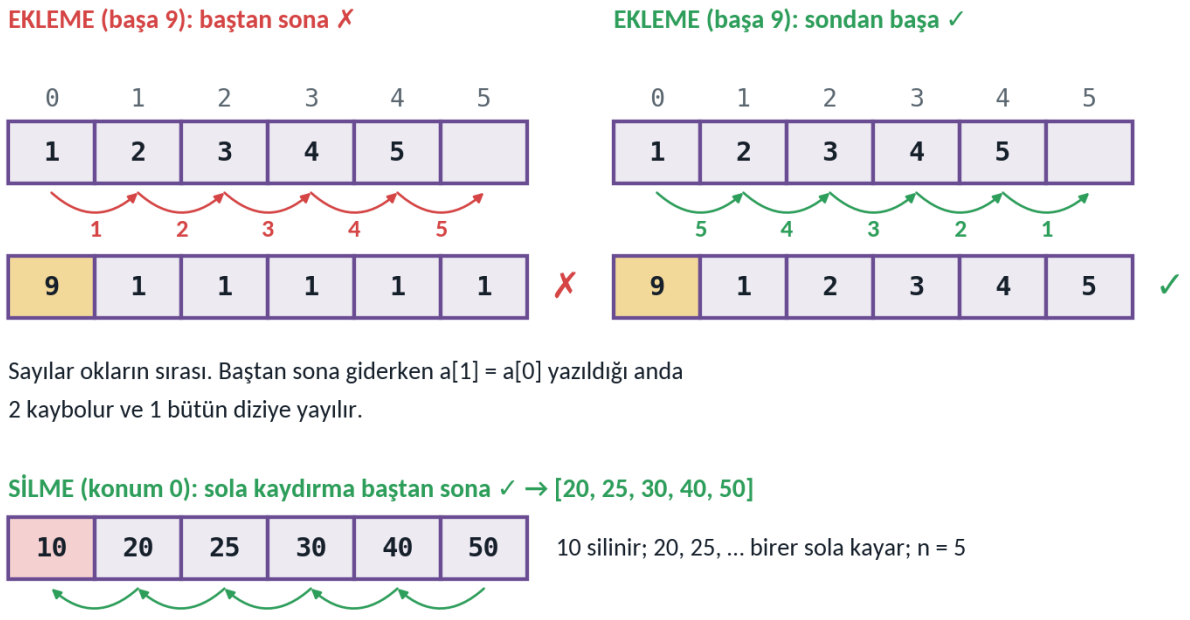
**H10_12_ekle_sil.c · satır 52–61**

```
52  int sil(int a[], int n, int konum)
53  {
54      if (konum < 0 || konum >= n) {
55          return -1;
56      }
57      for (int i = konum; i < n - 1; i++) {   /* BAŞTAN SONA: sola kaydır */
58          a[i] = a[i + 1];
59      }
60      return n - 1;
61  }
```

**5. Açıklama.** Satır 41: denetim **önce** yapılır; yer yokken kaydırma `a[kapasite]`'ye, yani dizinin dışına yazardı (Örnek 13'ün sorunu). Ekleme döngüsünün ilk turunda `a[n] = a[n - 1]`: `a[n]` dizinin içindedir, çünkü n < kapasite denetlendi. **6. Kaydırma yönünün etkisi** (gerçek çıktı):

`H10_K08_kaydirma_yonu.c` başa 9 eklerken iki yönü dener; gerçek çıktı: "Yanlış yön (baştan sona): 9 1 1 1 1 1", "Doğru yön (sondan başa) : 9 1 2 3 4 5".

**Şekil 10.4**
*Kaydırma yönü: eklemede sondan başa, silmede baştan sona; okların üstündeki sayılar atama sırası*




Sayılar okların sırası. Baştan sona giderken a[1] = a[0] yazıldığı anda 2 kaybolur ve 1 bütün diziye yayılır.




**7. Örnek çalıştırma.** Programın gerçek çıktısı altı satırdır; ilk satırı "başta : [10, 20, 30, 40, 50] n = 5", diğerleri Tablo 10.6'dedir.

**8. Test tablosu** (Tablo 10.6):

**Tablo 10.6**
*Örnek 12'nin test tablosu (H10_12_ekle_sil.c'nin gerçek çıktısı)*

| Çağrı | Durum | Gerçek sonuç |
|---|---|---|
| ekle(2, 25) | normal: ortaya | `[10, 20, 25, 30, 40, 50]  n = 6` |
| sil(0) | sınır: ilk eleman | `[20, 25, 30, 40, 50]  n = 5` |
| ekle(n, 60) | sınır: sona (konum = n) | `[20, 25, 30, 40, 50, 60]  n = 6` |
| ekle(0, 5) | hatalı: dizi dolu | `-1 (dizi dolu)` |
| sil(7) | hatalı: konum ≥ n | `-1 (geçersiz konum)` |

**9. Değiştir–genişlet:** (a) Sıralı bir diziye, sırayı bozmadan eklenecek konumu doğrusal taramayla bulun (ekleme sıralamasının bir adımı; AP-2). (b) Belirli bir **değeri** (konumu değil) silen `deger_sil` yazın. (c) Tekrar eden elemanları silin (Alıştırma 10.11).

## 4.7 Örnek 13 ★★★ — sınır dışı yazma deneyi

`H10_13_sinir_disi.c`, `int a[4]`'ün iki yanına iki değişken koyar ve kullanıcıdan gelen indise **denetimsiz** yazar. Dosya bilerek hatalıdır ama `-Wall -Wextra -pedantic` ile, hatta `-O2` ile bile **hiçbir uyarı** vermez: indis çalışma zamanında belli olur.

**H10_13_sinir_disi.c · satır 20–22**

```
20      int once = 111;                    /* dizinin komşuları */
21      int a[N] = {10, 20, 30, 40};
22      int sonra = 222;
```

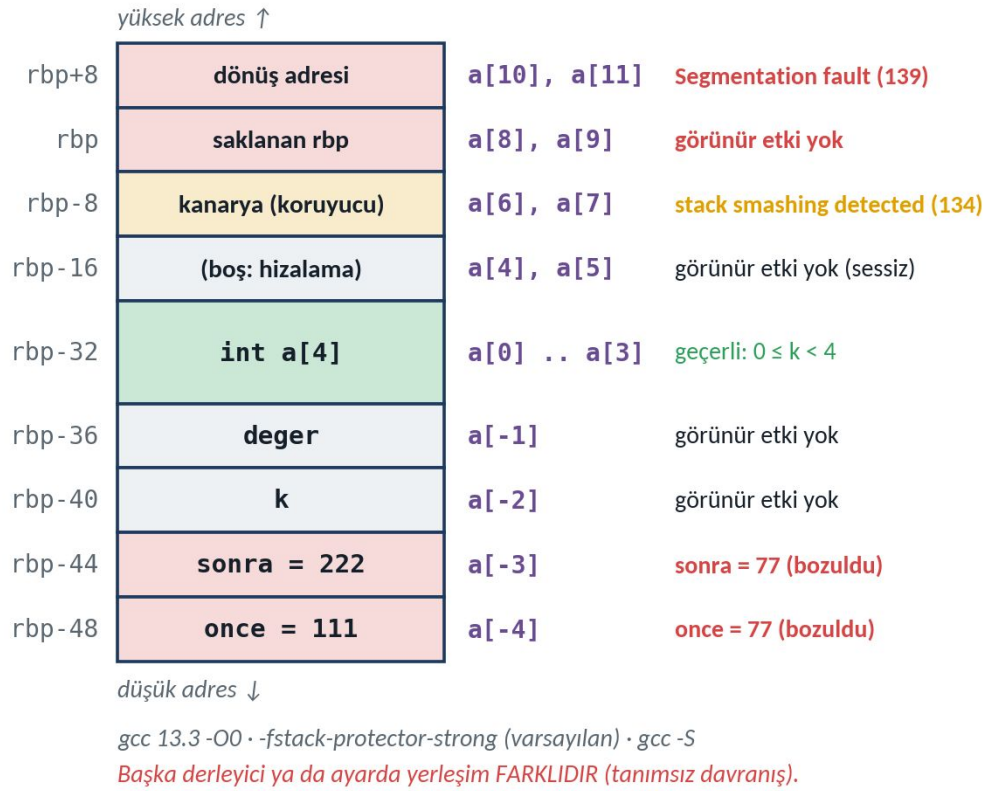
**H10_13_sinir_disi.c · satır 35**

```
35      a[k] = deger;                      /* SINIR DENETİMİ YOK */
```

**GERÇEK ÇALIŞTIRMALAR, DEĞER 77 (HER SATIR BİR ÇALIŞTIRMANIN SON SATIRI; TAMAMI KODLAR/CİKTİLAR/ ALTINDA)**

```
k =  2 → a = {10, 20, 77, 40} once = 111 sonra = 222
k =  4 → a = {10, 20, 30, 40} once = 111 sonra = 222
k =  5 → a = {10, 20, 30, 40} once = 111 sonra = 222
k = -4 → a = {10, 20, 30, 40} once = 77 sonra = 222
k = -3 → a = {10, 20, 30, 40} once = 111 sonra = 77
k =  6 → a = {10, 20, 30, 40} once = 111 sonra = 222 · *** stack smashing detected
*** · çıkış 134
```

**Şekil 10.5**
*main'in yığın çerçevesi ve sınır dışı yazmanın gözlenen etkileri (gcc 13.3, -O0, varsayılan yığın koruyucusu)*




*gcc 13.3 -O0 · -fstack-protector-strong (varsayılan) · gcc -S*
*Başka derleyici ya da ayarda yerleşim FARKLIDIR (tanımsız davranış).*

Gözlemler, **bu derleyici ve bu ayarlar için** şöyledir: `a[4]` ve `a[5]`'e yazmak görünürde **hiçbir şey** değiştirmedi; `a[-4]` `once`'yi, `a[-3]` `sonra`'yı bozdu; `a[6]` ise program sonunda `*** stack smashing detected ***` iletisiyle durdurulmasına yol açtı (çıkış durumu 134 = 128 + SIGABRT). Nedeni yığın çerçevesinin yerleşimidir: `gcc -S` çıktısına göre `once` rbp − 48, `sonra` rbp − 44, `k` ve `deger` rbp − 40 ve − 36'da, dizi rbp − 32 … − 17'de; dizinin hemen üstünde 8 baytlık boş bir hizalama bölgesi, onun üstünde de **yığın koruyucusunun** (stack protector) rbp − 8'e koyduğu "kanarya" (canary) değeri vardır (Şekil 10.5). Bu GCC'de `-fstack-protector-strong` varsayılan olarak açıktır (`gcc -Q --help=common`): fonksiyon dönmeden kanaryanın değişip değişmediği denetlenir ve değiştiyse program durdurulur (Bryant & O'Hallaron, 2016). Aynı kaynak `-fno-stack-protector` ile derlendiğinde yerleşim değişti: bu kez `a[-1]` `sonra`'yı bozdu. Sonuç: hangi komşunun bozulacağı **derleyiciye, seçeneklere ve platforma bağlıdır**; standart açısından hepsi tanımsız davranıştır.

**Yakalamak.** AddressSanitizer (ASan), her yerel dizinin çevresine yasak bölgeler (redzone) koyar ve her erişimi denetler (Serebryany vd., 2012). `-fsanitize=address` ile derlenen sürüm, `4 77` girdisinde **ilk** sınır dışı yazmada durur ve tam yerini söyler:

GERÇEK ASAN RAPORU (KISALTILMIŞ; TAMAMI KODLAR/CİKTİLAR/H10_13_SİNİR_DİSİ_ASAN_TAM.TXT)

```
$ gcc -std=c17 -g -fsanitize=address H10_13_sinir_disi.c -o sinir_disi_asan
$ echo '4 77' | ./sinir_disi_asan
==21652==ERROR: AddressSanitizer: stack-buffer-overflow
WRITE of size 4
    #0 in main H10_13_sinir_disi.c:35
    #0 in main H10_13_sinir_disi.c:19
    [96, 112) 'a' (line 21) <== Memory access at offset 112 overflows this variable
```

**Güvenlik tartışması.** Bu deneyde çoğunlukla bir sayı bozuldu. Aynı çerçevede kanaryanın üstünde saklanan `rbp` ve **dönüş adresi** durur (Şekil 10.5, a[8]–a[11]): `a[10] = 77` programı `main`'den dönerken Segmentation fault ile çökertti (çıkış 139; −6…12 arasındaki her indisin gerçek sonucu `ciktilar/H10_13_sinir_disi_tum_indisler.txt`'de). Dönüş adresini kullanıcıdan gelen veriyle ezebilen bir saldırgan, fonksiyon döndüğünde programın akışını saptırabilir. Yığın koruyucusu, ASLR ve yürütülemez yığın bu saldırıları **zorlaştıran** savunmalardır, ama hiçbiri sınır denetiminin yerini tutmaz (Seacord, 2013). Tampon taşmaları H13'te karakter dizileriyle yeniden karşımıza çıkacak.

## 4.8 Örnek 14 ★★★ — Eratosthenes Kalburu (meydan okuma)

**H10_14_eratosthenes.c · satır 34–40**

```
34      long isaretleme = 0;
35      for (int p = 2; p <= ust / p; p++) {        /* p * p <= ust */
36          if (asal[p]) {
37              for (int kat = p * p; kat <= ust; kat += p) {
38                  asal[kat] = false;      /* p'nin katı: asal değil */
39                  isaretleme++;
40              }
```

Girdi 100 için program 2'den 97'ye 25 asalı onarlı satırlarla yazar ve "2..100 arasında 25 asal (104 işaretleme)" der (gerçek çıktı).

`bool asal[i]` dizisi "i asal mı?" sorusunun yanıtını tutar; değer yine **indis** olarak kullanılır. Hepsi `true` başlar; her asal p için $p^2$, $p^2 + p$, … katları `false` yapılır: p'nin $p^2$'den küçük katlarının küçük bir çarpanı vardır ve zaten işaretlenmiştir. Dış döngü `p <= ust / p` ile durur (H7'deki gibi `p * p` taşmasın diye). Kalburda hiç bölme yoktur.

**Tablo 10.7**
*Aynı aralıklar için H7'nin √n testi (H07_11_asal_karekok_sure.c) ve kalbur: gerçek işlem sayıları*

| Üst sınır | Asal sayısı | H7 √n yöntemi: bölme | Kalbur: işaretleme |
|---|---|---|---|
| 1 000 | 168 | 2 351 | 1 411 |
| 10 000 | 1 229 | 55 958 | 16 981 |
| 100 000 | 9 592 | 1 345 442 | 193 078 |

İki yöntem aynı asal sayılarını buldu. Kalbur, her üst sınırda daha az işlem yaptı ve bu işlemler bölmeden çok daha ucuz olan dizi yazmalarıdır; bedeli `ust + 1` baytlık bir dizidir (bellek ↔ zaman takası). Kalbur, tek bir sayının asallığı için değil, **bir aralıktaki bütün** asallar için uygundur. Algoritmanın biçimsel çözümlemesi AP-2'nin konusudur.

# 5. Hata Avcısı

Her hatalı program `kodlar/hata_avcisi/` altında bir **düzeltilmiş** eşiyle birlikte durur; iletiler GCC 13.3'ün gerçek çıktılarıdır (Tablo 10.8).

**Tablo 10.8**
*Hata Avcısı özeti*

| Hata | Belirti (gerçek) | Neden | Çözüm |
|---|---|---|---|
| HA1 · a[n] (bir-fark) | -O0: uyarı yok; Toplam = 33134 (beklenen 369) | i <= N: son turda notlar[N] okunur | i < N; geçerli indisler 0..N-1 |
| HA2 · ilk değersiz dizi | yüz 1: 1002 kez | yerel dizi belirsiz değerlerle başlar | int frekans[7] = {0}; |
| HA3 · fonksiyonda sizeof | -Wsizeof-array-argument; n = 2 | parametre int * (8 bayt) | n'yi ayrı parametre yap |
| HA4 · = ve == ile dizi | error: assignment to expression with array type; -Warray-compare | dizi adı atanamaz; == adres karşılaştırır | eleman eleman döngü |
| HA5 · scanf("%d", a[i]) | -Wformat; Segmentation fault | değer adres sanılır | scanf("%d", &a[i]) |

**HA1 — a[n] (bir-fark hatası).** `H10_HA1_bir_fark.c` beş notu `for (int i = 0; i <= N; i++)` döngüsüyle toplar.

GERÇEK: ÇÖP TOPLAM, -O2 UYARISI, ASAN

```
$ ./ha1
Toplam = 33134 (beklenen 369)
$ gcc -std=c17 -Wall -Wextra -pedantic -O2 H10_HA1_bir_fark.c -o ha1_O2
19:25: warning: iteration 5 invokes undefined behavior [-Waggressive-loop-
optimizations]
18:23: note: within this loop
$ ./ha1_asan
READ of size 4
SUMMARY: AddressSanitizer: stack-buffer-overflow H10_HA1_bir_fark.c:19 in main
```

`-O0` ile derleyici hiçbir şey söylemedi ve program dizinin hemen ardındaki 4 baytı toplama kattı. `-O2` ile derleyici döngüyü çözümlerken 5. turun tanımsız davranış olduğunu fark edip uyardı; bu uyarıya güvenmeyin: indis girdiden gelseydi bunu da göremezdi (Örnek 13). ASan okumayı yakaladı.

**HA2 — ilk değeri olmayan dizi.** `H10_HA2_ilk_deger_yok.c`'de `zar_say` altı atışı `int frekans[7];` dizisinde sayar, ama `= {0}` unutulmuştur; ondan önce çağrılan `onceki_is` kendi yerel dizisine 1000, 1001, … yazıp döner.

Gerçek çıktı (-O0): "önceki iş bitti (son değer 1007)", "yüz 1: 1002 kez", "yüz 2: 1003 kez", "yüz 3: 1004 kez", "yüz 4: 1004 kez", "yüz 5: 1005 kez", "yüz 6: 1009 kez".

`-O0` sürümünde sayaçlar 1000'lerden başladı: `zar_say`'ın çerçevesi, az önce dönen `onceki_is`'in yığında bıraktığı değerlerin (1000, 1001, …) üstüne kuruldu (H9: çerçeveler aynı bölgeyi yeniden kullanır). Aynı kaynağın `-O2` ile derlenmiş sürümü ise bu kez "doğru" sonuç verdi (yüz 6: 3 kez; çıktı `kodlar/hata_avcisi/ciktilar/`'da): belirsiz bir değer **şansa** bağlıdır ve bir derlemede doğru görünmesi hiçbir şey kanıtlamaz. Derleyici iki durumda da uyarmadı.

**HA3 — fonksiyon içinde sizeof.** `H10_HA3_sizeof_parametre.c`'deki `double ortalama(int dizi[])` eleman sayısını `int n = (int)(sizeof(dizi) / sizeof(dizi[0]));` ile bulmaya çalışır:

GERÇEK: UYARI VE YANLIŞ SONUÇ

```
26:25: warning: ‘sizeof’ on array function parameter ‘dizi’ will return size of ‘int
*’ [-Wsizeof-array-argument]
$ ./ha3
main     : sizeof(notlar) = 24 bayt -> 6 eleman
ortalama : sizeof(dizi) = 8 bayt -> n = 2
Ortalama = 78.50 (beklenen 74.33)
```

`main`’de `sizeof(notlar)` 24 bayttır (6 eleman); fonksiyonda `sizeof(dizi)` 8’dir: parametre `int *`’dır. Fonksiyon n’yi 2 sanıp yalnızca ilk iki notun ortalamasını (78,50) aldı. Kural: eleman sayısı dizinin bildirildiği yerde hesaplanır ve **ayrı parametre** olarak verilir (düzeltilmiş dosya: “Ortalama = 74.33”).

**HA4 — = ve == ile dizi.** `H10_HA4_dizi_kopyalama.c` bir diziyi `yedek = notlar;` ile kopyalamaya, `H10_HA4_dizi_esitlik.c` iki eşit diziyi `a == b` ile karşılaştırmaya çalışır:

GERÇEK DERLEYİCİ İLETİLERİ VE ÇIKTI

```
yedek = notlar; → 16:11: error: assignment to expression with array type
16:11: warning: comparison between two arrays [-Warray-compare]
   16 |     if (a == b) {                      /* HATA: adres karşılaştırması */
$ ./ha4b
a ve b farklı (içerikleri aynı olduğu hâlde!)
```

Dizinin adı değiştirilebilir bir değer (lvalue) değildir; atama derleme hatasıdır. `a == b` ise derlenir ama iki ayrı dizinin **adreslerini** karşılaştırır ve hep “farklı” der; GCC 13 bunu `-Warray-compare` ile uyarır ve adresleri karşılaştırmak istiyorsanız `&a[0] == &b[0]` yazmanızı önerir. Çözüm: eleman eleman döngü (Bölüm 3.7, `H10_HA4_dizi_duzeltilmis.c`).

**HA5 — scanf’te & unutulması.** `H10_HA5_scanf_adres.c` sıfırla başlatılmış `int a[3]`’e `scanf("%d", a[i]);` ile okur:

GERÇEK: UYARI VE ÇÖKÜŞ

```
18:17: warning: format ‘%d’ expects argument of type ‘int *’, but argument 2 has type
‘int’ [-Wformat=]
3 sayı: 5 6 7
bash: … Segmentation fault      ./ha5
çıkış durumu: 139
```

`a[i]` elemanın **değeridir** (0); `scanf` bu değeri adres sanıp 0 adresine yazmaya çalıştı ve işletim sistemi programı durdurdu (Segmentation fault, çıkış 139 = 128 + 11). `-Wall` ile gelen `-Wformat` uyarısı hatayı derleme anında gösteriyordu: **uyarıları hata gibi okuyun**.

## 6. Mühendis Gözüyle: Sınırı Programın Kendisi Bilmeli

- **Her dış indisi denetleyin.** Kullanıcıdan, dosyadan ya da ağdan gelen bir indis veya uzunluk, diziye erişmeden önce `0 <= i && i < n` ile denetlenir. SEI CERT C kuralı ARR30-C, dizinin dışını gösteren indis ve işaretçilerin oluşturulmamasını ve kullanılmamasını ister (SEI, 2016). Heartbleed tam olarak bu kuralın ihlaliydi.
- **Kapasite tek yerde, doluluk ayrı değişkende.** `#define KAPASITE 100` ve `int n;` her ekleme öncesinde `n < KAPASITE` sorulur (Mini-OBS’nin “Kapasite dolu” iletisi).
- **Sınır testleri yazın.** n = 0, 1, kapasite − 1, kapasite; ilk ve son konum; bulunamayan değer. Örnek 7 ve 12’nin test tabloları bu yüzden var.

- **Araçlar.** Geliştirirken `-Wall -Wextra` ve `-fsanitize=address,undefined` ile derleyin; ASan sınır dışı erişimi **çalışma anında** yakalar (Serebryany vd., 2012). Bu haftanın bütün temiz örnekleri ve Mini-OBS testleri bu denetimden geçti.
- **Taşınabilirlik ve okunabilirlik.** `int`'in boyutu platforma bağlıdır: sabit yazmak yerine `sizeof` kullanın; VLA MSVC'de yoktur; büyük diziler yığını taşırabilir (H9). Salt okunan parametreye `const` koymak sözleşmeyi derleyiciye de söyler.

# 7. Mini-OBS Adımı: v0.10 — Paralel Dizilerde Kayıt, İstatistik ve Histogram

v0.9 öğrencileri saklamadan istatistik güncelliyordu (`static` akış değişkenleri). v0.10'da kayıtlar `main.c`'deki dört **paralel dizide** saklanır: `no[]`, `vize[]`, `final[]`, `ort[]` (kapasite 100, dolu eleman sayısı `n`). İstatistikler her istendiğinde dizilerden yeniden hesaplanır; menü beş seçeneklidir. `obs.c`'deki fonksiyonlar diziyi ve `n`'yi parametre olarak alır (`numara_bul(no, n, x)`, `dizi_ortalama(ort, n)`, `harf_dagilimi(ort, final, n, sayac)` …).

**Gereksinimler.** (1) Not girişi: kapasite doluysa ileti verilir; aynı öğrenci numarası ikinci kez girilemez (`numara_bul` ile doğrusal tarama). (2) İstatistikler: öğrenci sayısı, sınıf ortalaması, **popülasyon standart sapması**, en yüksek ve en düşük (numarasıyla), ortalamanın üstündeki öğrenci sayısı, başarılı (CC ve üstü) ve şartlı (DC) sayıları. (3) Kayıtları tablo olarak listeleme. (4) Harf notu dağılımı histogramı. (5) Kayıt yokken 2, 3 ve 4 "Henüz kayıt yok" der; girdi biterse program 1 ile çıkar. Kurallar (RTEÜ tablosu, final barajı 50, şartlı DC) değişmedi.

**Standart sapma elle (T1 testi).** `obs.c`'deki `standart_sapma(const double a[], int n)` iki geçişlidir (Örnek 3 gibi): önce `dizi_ortalama` ile μ, sonra `kare_toplam += fark * fark;` ile sapmaların kareleri; sonuç `sqrt(kare_toplam / n)`, yani $\sigma = \sqrt{\Sigma(x_i - \mu)^2 / n}$ (popülasyon: n'ye bölünür). Beş ortalama 79,00; 46,80; 93,00; 58,00; 67,20 için μ = 68,80:

**Tablo 10.9**
*Standart sapmanın elle hesabı: varyans 1292,88 / 5 = 258,5760; σ = √258,5760 ≈ 16,08 (program: "Standart sapma : 16.08")*

| Öğrenci no | ortalama $x_i$ | $x_i - \mu$ | $(x_i - \mu)^2$ |
|---|---|---|---|
| 2301010045 | 79,00 | 10,20 | 104,0400 |
| 2301010046 | 46,80 | -22,00 | 484,0000 |
| 2301010047 | 93,00 | 24,20 | 585,6400 |
| 2301010048 | 58,00 | -10,80 | 116,6400 |
| 2301010049 | 67,20 | -1,60 | 2,5600 |
| Toplam | | | 1292,8800 |

`sqrt` için `<math.h>` ve bağlarken `-lm` gerekir; unutulursa bağlayıcı "undefined reference to `sqrt`" der (gerçek ileti `mini_obs_v0_10_ciktilar.txt`'de). Ortalamanın üstü **kesin büyük** sayılır (>).

**Kabul ölçütleri.** Dört kaynak dosya sıfır uyarıyla derlenir (`make`); `make test` 53 doğrulamanın (37 kural + 16 dizi) hepsini geçer; aşağıdaki yedi senaryonun çıktısı beklenenle aynıdır; ASan + UBSan ile `obs_test` ve bütün senaryolar temiz çalışır.

**Tablo 10.10**
*Mini-OBS v0.10 test senaryoları (91_Proje_MiniOBS/v0.10/testler; gerçek çalıştırmalar)*

| Test | Senaryo | Gerçek sonuç (özet) | Çıkış |
|---|---|---|---|
| T1 | 5 öğrenci; 2, 3, 4 | ort 68.80, σ 16.08 | 0 |
| T2 | aynı numara iki kez | "Bu numara zaten kayıtlı (sıra 1)." | 0 |
| T3 | geçersiz menü, numara ve not | geçersiz girdiler yeniden soruldu | 0 |
| T4 | kayıt yokken 2, 3, 4 | "Henüz kayıt yok" (×3) | 0 |
| T5 | not girişinde EOF | "Girdi sona erdi." | 1 |
| T6 | sınır değerler (59.5, 49.9 …) | ort 66.98, σ 14.96 | 0 |
| T7 | 100 öğrenci + 101. deneme | ort 72.64, σ 11.25 | 0 |

**Bilinçli sınırlar.** Sabit kapasite (100) → H12'de dinamik bellek (`malloc`); paralel dizilerin eşgüdüm yükü → H14'te `struct Ogrenci`; tek ders → H11'de öğrenci × ders not **matrisi** (v0.11).

# 8. Alıştırmalar ve Atölye

Alıştırmalardaki kodlar `kodlar/alistirmalar/` altındadır; çözümler öğretim üyesindedir.

**Alıştırma 10.1** ★☆☆ *Çıktı tahmini*

Program iki satır yazdırır. Çalıştırmadan tahmin edin; `a[a[2]]` için önce iç indisi hesaplayın (`H10_A01_cikti_tahmini.c`).

**alistirmalar/H10_A01_cikti_tahmini.c · satır 8–20**

```c
int main(void)
{
    int a[6];
    for (int i = 0; i < 6; i++) {
        a[i] = i * i;
    }
    printf("%d %d %d\n", a[2], a[a[2]], a[5] - a[4]);

    int b[5] = {4};
    int n = (int)(sizeof(b) / sizeof(b[0]));
    printf("%d %d\n", b[0] + b[4], n);
    return 0;
}
```

### Alıştırma 10.2 ★★☆ *Çıktı tahmini*

Birinci bölüm diziye ne yapıyor? İkinci bölümdeki `say[]` neyi sayıyor? İki satırı tahmin edin (`H10_A02_cikti_tahmini_kaydirma.c`).

**alistirmalar/H10_A02_cikti_tahmini_kaydirma.c · satır 8–28**

```c
int main(void)
{
    int a[5] = {1, 2, 3, 4, 5};
    int son = a[4];
    for (int i = 4; i > 0; i--) {
        a[i] = a[i - 1];
    }
    a[0] = son;
    for (int i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    int veri[] = {2, 0, 2, 1, 2, 0};
    int say[3] = {0};
    for (int i = 0; i < 6; i++) {
        say[veri[i]]++;
    }
    printf("%d %d %d\n", say[0], say[1], say[2]);
    return 0;
}
```

### Alıştırma 10.3 ★★☆ *Hata bul*

Üç notun ortalaması 80.00 olmalıyken program 56.00 yazıyor. **İki** hatayı bulun; hangisi bir-fark hatasıdır? Neden program çökmedi? (`H10_A03_hata_bul_ortalama.c`)

**alistirmalar/H10_A03_hata_bul_ortalama.c · satır 13–20**

```c
    int notlar[KAPASITE] = {70, 80, 90};    /* kalanlar 0 */
    int n = 3;                              /* dolu eleman sayısı */
    int toplam = 0;
    for (int i = 1; i <= n; i++) {
        toplam += notlar[i];
    }
    double ortalama = toplam / n;
    printf("Ortalama: %.2f\n", ortalama);
```

### Alıştırma 10.4 ★★☆ *Hata bul*

`basa_ekle(a, 4, 0)` sonrası dizi `0 10 20 30 40` olmalı; program `0 10 10 10 10` yazıyor. Hata nerede? (`H10_A04_hata_bul_ekleme.c`)

**alistirmalar/H10_A04_hata_bul_ekleme.c · satır 10–17**

```c
int basa_ekle(int a[], int n, int deger)
{
    for (int i = 1; i <= n; i++) {
        a[i] = a[i - 1];
    }
    a[0] = deger;
    return n + 1;
}
```

**Alıştırma 10.5** ★★☆ *Parsons*

Satırları, dizi küçükten büyüğe (azalmayan) sıralıysa `true` döndüren `sirali_mi` fonksiyonuna dönüşecek biçimde sıralayın ve girintileyin. **Bir satır fazladır**; hangisi ve neden? (`H10_A05_parsons_sirali_mi.c`)

**alistirmalar/H10_A05_parsons_sirali_mi.c · satır 13–21**

```
13          return false;
14      return true;
15  bool sirali_mi(const int a[], int n)
16      for (int i = 0; i < n; i++) {
17  }
18          if (a[i] > a[i + 1]) {
19  {
20      for (int i = 0; i < n - 1; i++) {
21          }
```

**Alıştırma 10.6** ★★☆ *Kod tamamla*

Dört boşluğu doldurun: program 0–10 arası puanların frekansını sayıp yalnızca en az bir kez geçenleri yazsın (beklenen: `3: 1`, `5: 1`, `7: 3`, `8: 1`, `10: 2`) (`H10_A06_kod_tamamla_frekans.c`).

**alistirmalar/H10_A06_kod_tamamla_frekans.c · satır 14–17**

```
14      int frekans[____] = {____};
15      for (int i = 0; i < n; i++) {
16          frekans[____]++;
17      }
```

**Alıştırma 10.7** ★☆☆ *Kod yaz*

n (1–100) ve n tamsayı okuyun; negatiflerin **sayısını**, **toplamını** ve **ilk** negatifin indisini (yoksa −1) yazdırın. Test: `6  4 -2 7 -5 0 -1` → 3 negatif, toplam −8, ilk indis 1; `3  1 2 3` → ilk indis −1.

**Alıştırma 10.8** ★★☆ *Kod yaz*

Diziyi k konum **sağa** döndüren `void saga_dondur(int a[], int n, int k)` yazın; k ≥ n olabilir (kaç tur yeter?). Kaydırma hangi yönde yapılmalı? Test: {1, …, 6} için k = 1 → `6 1 2 3 4 5`, k = 6 → değişmez, k = 8 → `5 6 1 2 3 4`.

**Alıştırma 10.9** ★★☆ *Kod yaz*

İki diziyi üçüncü bir dizide **birleştirin** (önce a, sonra b) ve a'nın b'de de geçen elemanlarını doğrusal taramayla yazdırın. Birleşik dizinin boyutu ne olmalı? Test: a = {3, 4, 7, 9}, b = {9, 1, 4} → birleşik `3 4 7 9 9 1 4`, ortak `4 9`.

**Alıştırma 10.10** ★★☆ *Kod yaz*

0–100 arası notlardan oluşan bir dizide **en sık geçen notu** (mod) bir frekans dizisiyle bulun; eşitlikte küçük notu seçin. Frekans dizisi kaç elemanlı olmalı? Test: {85, 70, 90, 70, 55, 85, 100, 70, 85, 40} → 70 (3 kez; 85 de 3 kez geçer ama 70 daha küçük).

**Alıştırma 10.11** ★★★ *Kod yaz*

`int tekrarsiz(int a[], int n)`: tekrar eden elemanları, **ilk geçişleri** ve sırayı koruyarak yerinde silin ve yeni n'yi döndürün (Örnek 12'nin `sil`'ini kullanabilirsiniz). Dikkat: silme sonrası aynı konuma yeni bir eleman gelir. Test: {4, 1, 4, 4, 7, 1, 9, 7} → `4 1 7 9`, n = 4.

**Alıştırma 10.12** ★★★ *Kod yaz*

`H10_13_sinir_disi.c`'yi güvenli hâle getirin: `bool guvenli_yaz(int a[], int n, int k, int deger)` yalnızca 0 ≤ k < n ise yazsın ve `true` döndürsün; değilse hiçbir şey yazmadan `false` döndürsün ve kullanıcı uyarılsın. `4 77`, `-4 77` ve `2 77` girdileriyle deneyin; ASan ile derleyip hiçbir rapor çıkmadığını gösterin. Heartbleed'in düzeltmesiyle benzerliği nedir?

ALGORİTMA ATÖLYESİ · **Atölye etkinlikleri (40 dk, eşli)**

- **Dizi desen kartları (15 dk).** Her eş bir kart çeker (toplam/ortalama, sayma, en büyük + indis, doğrusal tarama, iki uç, frekans) ve kartın çekirdek satırıyla bir mini problem yazar: en düşüğü atıp ortalama, 50 altını sayma, en küçüğün indisi, numara arama, palindrom, 0–10 puan dağılımı; n = 0 ve n = 1 ile dener.
- **Histogram atölyesi (10 dk).** `H10_09_zar_histogram.c`'de `ATIS`'ı 60, 600 ve 6000 yapıp tohumsuz derleyin (ya da web'deki histogram oluşturucuyu kullanın) ve payların %16,7'ye yaklaşmasını tablolayın; `H10_10_harf_dagilimi.c`'ye 10 vize–final çifti ekleyip dağılımı yorumlayın. Notlar 0–100 ise frekans dizisi kaç elemanlı olmalı?
- **Mini-OBS'nin dizilere taşınması (15 dk).** v0.9'un akış istatistiklerini dört paralel diziye taşıyın: önce `numara_bul` ve `dizi_ortalama`, sonra eklemeden önce kapasite denetimi; T1 ve T7 ile doğrulayın (Bölüm 7).

## 9. Kendini Değerlendirme

- ☐ Bir diziyi bildirip tam, kısmi ve `{0}` ilk değer listeleriyle başlatabiliyorum; yerel bir dizinin ilk değersiz elemanlarının belirsiz olduğunu biliyorum.
- ☐ `&a[i]` adresini taban + i × sizeof(eleman) ile hesaplayabiliyor; `sizeof(dizi) / sizeof(dizi[0])`'ın fonksiyon parametresinde neden çalışmadığını açıklayabiliyorum.
- ☐ Toplam, sayma, en büyük ve indisi, doğrusal tarama, ters çevirme, döndürme, ekleme/silme (doğru kaydırma yönüyle) ve frekans desenlerini yazabiliyorum.
- ☐ Diziyi eleman sayısıyla birlikte fonksiyona aktarıyor, salt okunan parametrelere `const` koyuyorum.
- ☐ Sınır dışı erişimin tanımsız davranış olduğunu, sonucunun derleyiciye bağlı olduğunu ve Heartbleed ile bağlantısını açıklayabiliyorum; ASan ile hatayı yakalayabiliyorum.

## 10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · **Bu haftanın yapı taşları AP-2'de nerede kullanılacak?**

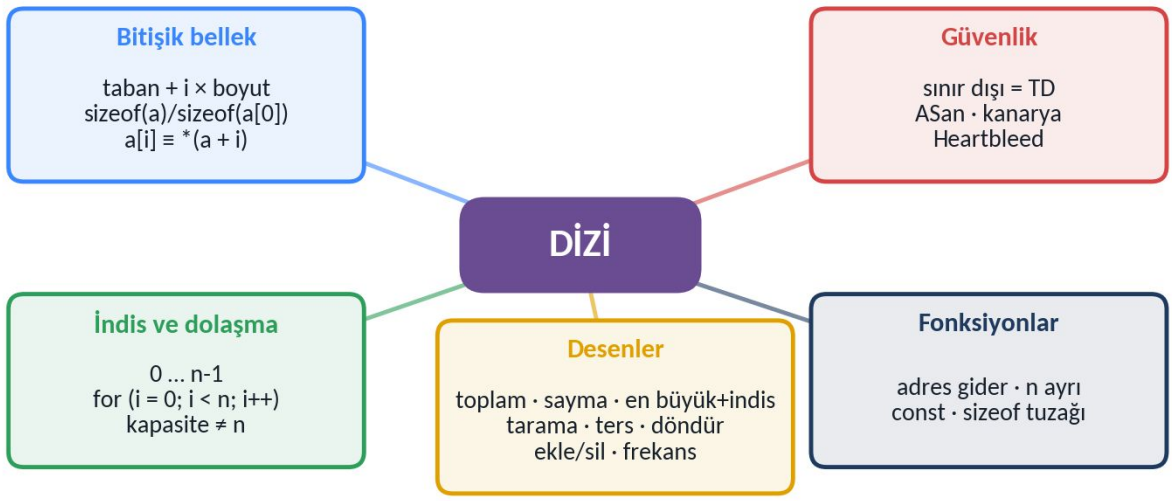
- Doğrusal tarama → doğrusal aramanın en iyi, en kötü ve ortalama durum çözümlemesi.
- En küçüğün indisini bulma + takas → seçme sıralamasının neredeyse tamamı.
- İki uç → iki işaretçi (two pointers) yöntemi; frekans dizisi → sayma sıralaması (counting sort).

> **MERAK KUTUSU · a[i] ile i[a] neden aynı şeydir?**
>
> C'de `a[i]`, `*(a + i)` demektir; toplama değişmeli olduğundan `i[a]` (ör. `2[a]`) de aynı elemanı verir. Gerçek kodda kullanmayın; ama dizi erişiminin "adres + uzaklık" olduğunu çok iyi gösterir (ayrıntı H12'de).

## 11. Özet, Kavram Haritası ve Terimler

- Dizi, aynı tipten değerleri tek adla ve **bitişik** bellekte tutar; i. elemanın adresi taban + i × sizeof(eleman)'dır, bu yüzden indisler 0'dan başlar ve son geçerli indis n − 1'dir.
- Kapasite derleme zamanı sabitidir; dolu eleman sayısı ayrı bir değişkende tutulur. `sizeof(a)/sizeof(a[0])` yalnızca bildirimin olduğu yerde çalışır.
- Fonksiyona dizinin adresi gider: elemanlar değiştirilebilir, boyut ayrı parametredir, salt okunan dizi `const`'tur. Desenler: toplam, sayma, en büyük + indis, doğrusal tarama (erken çıkış, −1), iki uç (ters çevirme), döndürme, kaydırmayla ekleme/silme (yön!), frekans dizisi (değer = indis).
- Sınır dışı erişim tanımsız davranıştır: sessiz bozulma, çökme ya da "çalışıyor" görünme; Heartbleed eksik bir uzunluk denetimiydi. Denetim programın işidir; ASan geliştirirken yakalar.

**Şekil 10.6**
*Hafta 10 kavram haritası*




**Tablo 10.11**
*Terim sözlüğü (Bölüm 6.8 terim standardıyla uyumlu)*

| Türkçe | İngilizce | Türkçe | İngilizce |
|---|---|---|---|
| dizi | array | sınır dışı erişim | out-of-bounds access |
| indis | index | bir-fark hatası | off-by-one error |
| eleman | element | tanımsız davranış | undefined behavior |
| bitişik bellek | contiguous memory | tampon sınırı ötesini okuma | buffer over-read |
| taban adres | base address | tampon taşması | buffer overflow |
| ilk değer listesi | initializer list | yığın koruyucusu, kanarya | stack protector, canary |

| Türkçe | İngilizce | Türkçe | İngilizce |
|---|---|---|---|
| belirlenmiş ilk değer | designated initializer | frekans dizisi | frequency array |
| değişken uzunluklu dizi | variable-length array (VLA) | doğrusal tarama | linear search |
| paralel diziler | parallel arrays | önbellek satırı | cache line |
| kapasite | capacity | uzamsal yerellik | spatial locality |

> NOT · **Haftaya hazırlık: H11 — Diziler II: matrisler**
>
> İki boyutlu diziler (`int not[30][5]`) de bellekte **tek bir bitişik şerittir**: adres `taban + (i × sütun_sayısı + j) × boyut`. Mini-OBS v0.11 not matrisini ekleyecek.

# 12. Kaynaklar


Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Carnegie Mellon University Software Engineering Institute. (2016). *SEI CERT C coding standard: Rules for developing safe, reliable, and secure systems* (2016 ed.). Carnegie Mellon University.

Durumeric, Z., Li, F., Kasten, J., Amann, J., Beekman, J., Payer, M., Weaver, N., Adrian, D., Paxson, V., Bailey, M., & Halderman, J. A. (2014). The matter of Heartbleed. In *Proceedings of the 2014 Internet Measurement Conference* (pp. 475–488). ACM. https://doi.org/10.1145/2663716.2663755

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C programming: A modern approach* (2nd ed.). W. W. Norton.

Microsoft. (t.y.). *Predefined macros*. Microsoft Learn. https://learn.microsoft.com/en-us/cpp/preprocessor/predefined-macros

National Institute of Standards and Technology. (2014). *CVE-2014-0160 detail*. National Vulnerability Database. https://nvd.nist.gov/vuln/detail/CVE-2014-0160

OpenSSL Project. (2014, April 7). *OpenSSL security advisory: TLS heartbeat read overrun (CVE-2014-0160)*. https://www.openssl.org/news/secadv/20140407.txt

Seacord, R. C. (2013). *Secure coding in C and C++* (2nd ed.). Addison-Wesley.

Serebryany, K., Bruening, D., Potapenko, A., & Vyukov, D. (2012). AddressSanitizer: A fast address sanity checker. In *2012 USENIX Annual Technical Conference (USENIX ATC 12)* (pp. 309–318). USENIX Association.